

CS 30700: Sprint 2 Retrospective

BoilerSpace

Team 3: Aagam Bapna, Sheehan Dandapat, Rohan Potta, Nathan Wan, Jed Macapinlac, Bidit Acharyya

What went well?

- **User Story 10:** As a user, I want to update my availability or preferred study times, so that others know when I'm typically open to studying. This user story is integral when considering the social and timekeeping features within our web-app, as students deserve to know how to plan study times together with classmates as well as planning other things around those times. By achieving this, we were able to give more freedom to students in the form of finding friends based on shared times.
- **User Story 26:** As a student, I want to see a timestamp of the last update for a room's status so I can judge how reliable the information is before I start walking. This user story goes hand in hand with student well being, as students must know if a room is open as well as how accurate that information may be. Time finding a room to study in is always lost time, so prioritizing features like these ensure students can be more focused on work.
- **User Story 36:** As a student, I want to see a small icon next to a building name that indicates if it has a cafe or vending machines nearby so I can choose a study spot based on snack availability and have a way to crowdsource that data. When students study, they often need fuel to keep going, and this comes in the form of food or snacks. Buildings with these amenities are much more appealing to students, which is why this is an integral feature to let students know of such advantages.
- **User Story 42:** As a student, I want to see a list of rooms sorted by distance from my current location so that I can find the nearest available spot. This user story is important as it allows users to reference our app to find nearby spots on campus for impromptu study sessions. The app asks for the user's location data,

and uses the Haversine distance formula to compute the distances to each building, giving the most accurate data possible.

- **User Story 54:** As a student, I want to filter rooms based on amenities (whiteboards, outlets) so that I can find a workspace that fits my equipment needs. This user story is important as it allows users to be more informed when deciding where to study. The user is able to click on a building, select any combination of amenities they want in a room (e.g. whiteboard, outlets, etc), and then receive a filtered list of rooms in the building that meet this criteria.
- **User Story 57:** As a student, I want the AI assistant to generate concise study guides from my notes and highlight key topics, so I can review efficiently and know what to focus on. This user story is important as it is one of the key features of our app, allowing users to create their own study materials tailored to what they studied and free of external information they don't need. Users can click on any of the classes they are registered in and with all the notes that have been uploaded for that class, they can click a button to generate the study guide, which creates vector embeddings using the notes to give context to the LLM model (Gemini), creating a comprehensive study guide with key concepts, important formulas, and topics to review.
- **User Story 12:** This user story allows students to personalize their profiles so that classmates can recognize them more easily across the platform, whether in course directories, study sessions, or direct messages. Having a recognizable face tied to a name builds trust and encourages social interaction. Implementation went smoothly as the GCS upload pipeline and image validation logic worked reliably from the start, and we encountered no major issues integrating it with the existing profile editing flow.
- **User Story 9:** This user story enables students to build a network of study partners based on shared coursework, which is one of the core social goals of BoilerSpace. Being able to discover and friend classmates directly from a course directory makes it easy to find people to study with for specific classes. The implementation went well as the classmate discovery query and friend request

integration worked as expected with no significant blockers, and privacy controls for opted-out users functioned correctly from the first pass.

- **User Story 47:** This user story gives students the ability to ask clarifying questions or provide corrections on shared notes, turning static uploads into collaborative study resources. This fosters a stronger sense of community and improves the quality of shared content over time. Implementation went smoothly as the comment schema, enrollment-based access control, and cascade cleanup on note deletion all worked without major issues, and the inline comment UI integrated naturally into the existing notes layout.
- **User Story 85: Direct Messaging:** The Socket.io integration worked reliably from the start, with real-time message delivery functioning correctly for online participants. The conversation and message schemas came together cleanly, and the inbox UI populated with conversation history, avatars, and last message previews without major issues. Blocked user enforcement also worked correctly, preventing socket events from reaching users who had been blocked.
- **User Story 13: Account Deletion with 2FA:** The cascading deletion logic worked correctly, wiping all associated user data including messages, notes, bookmarks, check-ins, and friendships in one clean sweep. Session invalidation fired immediately after deletion so old tokens stopped working right away. The OTP email flow integrated smoothly with the existing authentication setup, and the confirmation modal requiring the user to type their email before deletion added a clean layer of protection against accidental deletions.
- **User Story 84: Delete Messages:** The MongoDB TTL index for auto-disappearing messages worked out of the box, automatically removing expired messages without needing a manual cleanup job. The soft-delete approach of replacing message content with a placeholder preserved thread structure correctly so conversations stayed readable even after deletions. Author-only deletion enforcement worked as expected from the first pass, correctly blocking non-authors from deleting messages they did not send.
- **User Story 55: AI Course Q&A:** The backend context aggregation worked out well, correctly pulling relevant note content so responses only came from course

material and there would be no responses that did not come from the notes/ The frontend note integration fit well into our existing interface as there was already note upload functionality. This allows for students to ask questions directly from their notes.

- **User Story 58: AI Practice Questions:** The generation of practice questions required some initial tweaking of prompting to improve the quality and relevance of the questions, but after refinement, the output became consistent and useful for studying. The feature integrated smoothly into the existing study flow as there were other options such as a Q&A and study guide creation. As a result, it made the study flow more natural for students.
- **User Story 73: Club Member Approval:** The approval workflow integrated cleanly into the existing club management system, allowing organizers to review and approve join requests without disrupting other features. Since the Organizer Dashboard and functionality for that already existed, the functionality needed for this user story was relatively straightforward. Backend routes and state handling worked as expected, correctly enforcing approval before access was granted.
- **User Story 17: Room Capacity Notifications:** The capacity notification system came together smoothly, with the User schema extension for notification preferences integrating cleanly into the existing data model. The backend trigger logic hooking into check in and check out events was a natural fit since those events were already well defined, making threshold evaluation straightforward to implement. The react notification bell component slotted into the front end without disrupting the existing UI elements and the GET /notifications endpoint correctly scoped alerts to the authenticated user as expected.
- **User Story 18: Location Access Explanation:** The custom location permission modal was implemented cleanly, appearing before the browsers native prompt and providing clear context to the user without friction. State management for tracking granted, denied, and pending permission states worked reliably, and the segregation of location-dependent features when access was denied function as intended. The prompt trigger logic integrated well with the map and room list

features, correctly firing only on first access and not-appearing on subsequent page loads once the user had already responded.

- **User Story 41: Directions to Study Locations:** The MapBox Directions API integrated smoothly into the existing campus map component, with walking mode configuration and route overlay rendering working as expected from the start. The “Get Directions” button was straightforward to wire into building detail popups and room cards, and the side panel for step-by-step directions fit naturally alongside the existing map layout. Edge case handling for unavailable location access was simplified by reusing the modal already built for User Story 18, this avoided redundant work and kept the user experience consistent.
- **User Story 34: Enhanced Dashboard and Room pages:** The amenity fields extended the Room schema without requiring significant restructuring, and the seeding script updates were straightforward to apply across existing room data. Recent building visit tracking hooked naturally into the existing check-in flow, making the “Recently Visited” dashboard section relatively simple to populate and render. Amenity icons and the last-updated timestamp displayed correctly on room cards and detail pages, with relative time formatting working as expected to give users a reliable sense of how current the occupancy data was.

What did not go well?

Overall, Sprint 2 was a great continuation from Sprint 1, with a majority of the key features of our project being implemented. Since we tackled more challenging parts of the project in this sprint, such as AI integration and note storage, we did encounter some difficulties along the way. In Sprint 1, we implemented a basic note upload feature, but because the notes were stored locally, only the user who uploaded the note could download it. We used Google Cloud Storage to fix this, and integrating this caused some parts of our project to break, requiring some rewriting. For implementing the AI study guide, we had to use the Gemini API, but this came with problems like handling unpredictable response formatting. Having done a majority of the difficult tasks

in Sprint 2, we are confident that Sprint 3 will be much smoother and that our final project after the conclusion of Sprint 3 will be something we can be proud of.

How should we improve?

1. Standardize local dev environments and adopt CI to create a single source of truth for test results.

We ran into recurring issues where tests would pass on one developer's machine but fail on another's. This caused confusion and wasted time because instead of working on features or actual bugs, we were debugging environment differences, things like Node versions, missing env variables, or dependency mismatches. Sometimes a test fix commit would work for one person but break it for someone else, which led to a cycle of back-and-forth fix commits.

Going forward, we should standardize our local environments by using something like an `.nvmrc` file to lock the Node version and keeping `.env.example` up to date. More importantly, we should set up a CI pipeline so that every push and pull request gets tested in a clean, consistent environment. That way we have a single source of truth for whether tests actually pass, and we stop wasting sprint time on 'works on my machine' problems.

2. Structure demos around acceptance criteria, not just feature walkthroughs.

During our sprint demo, the grader pointed out a few acceptance criteria that we didn't explicitly cover. The issue wasn't that the work wasn't done, it was that our demo was structured more like a general walkthrough of the feature rather than a deliberate walkthrough of each acceptance criterion. This meant some criteria were either skipped or only loosely shown, which made it harder for the grader to confirm they were met.

Going forward, we should build our demo script directly from the acceptance criteria checklist. Before the demo, we should map each criterion to a specific action we'll show and make sure every single one gets addressed. This way the demo becomes a clear, structured proof that the story is complete rather than just a casual showing of the feature working.

3. Coordinate UI layout and navigation flow across features to reduce demo friction.

Our app has a lot of distinct UI sections. Map, recently viewed rooms, room cards, building cards, chats, profiles, and more. During the demo, we ran into moments of stalling and awkward navigation because features built by different team members didn't flow together naturally. Someone would need to show their feature but had to click through or work around another member's UI to get there, which made the demo feel disjointed and cost us time.

The root issue is that we each built our sections somewhat independently without aligning on how a user would actually navigate between them. Going forward, we should reflect on our UI flow chart — essentially a rough map of how screens connect and where each feature lives. This would not only make demos smoother since we'd have a logical path to follow, but it would also improve the actual usability of the app for end users. Even a quick whiteboard session at the start of the sprint to sketch out navigation and layout would go a long way.

4. Order demo presentations so that related features are shown back-to-back.

Some of our user stories had direct dependencies or connections to each other. For example, some of Jed's user stories (User Story 18 and User Story 41) involved a major geolocation implementation, Bidit also had a story (User Story 42) that also tied into location. During the demo, these were presented separately, which meant the grader saw location-related functionality at disconnected points rather than as a cohesive flow. It made the demo feel a bit scattered even though the actual product worked well together.

This didn't affect the product itself, but it did affect how clearly we communicated our work. Going forward, we should look at the sprint's user stories before the demo and group presenters so that related or dependent features are shown back-to-back. A quick five-minute conversation before the demo to map out "your story feeds into mine, so let's go one after the other" would make the whole presentation feel more natural and help the grader see how features connect without us having to explain it.

5. Fill async wait times during demos with meaningful commentary instead of dead air.

Since our app relies heavily on asynchronous operations, things like awaiting API responses, sending emails, updating location, or rendering UI changes, there were moments during the demo where we'd trigger something and then just sit there waiting for it to finish. This created awkward silence that broke the flow of the presentation and made it feel less polished.

Going forward, we should treat those wait times as opportunities rather than dead spots. While something is loading or processing, the presenter could explain how the feature works under the hood, walk through a related acceptance criterion, point out a design decision, or preview what's about to happen on screen. Even something as simple as "while this email is sending, let me show you how the notification will look on the other end" keeps the demo moving and shows the grader we understand our own system. A little prep beforehand to identify which features have noticeable wait times and planning what to talk about during those gaps would make a big difference.